

BÖLÜM 1: YAPAY ZEKÂ TEMELLERİ

1.1. Yapay Zekâ (Artificial Intelligence - AI) Nedir?

Yapay zekâ, bilgisayar sistemlerinin normalde insan zekâsı gerektiren görevleri (görsel algılama, konuşma tanıma, karar verme ve dil çevirisi gibi) yerine getirebilme yeteneğidir. Geleneksel programlamada bilgisayara ne yapması gerektiğini adım adım (if-else ve for/while döngüleriyle) söyleriz. Yapay zekâda ise sistem, kuralları bizim doğrudan yazmamıza gerek kalmadan, karşılaştığı örnekler üzerinden kendi başına çıkarır.

1.2. Kısa Tarihçe ve Dönüm Noktaları

Yapay zekânın gelişimi doğrusal olmamış, donanım yetersizlikleri nedeniyle zaman zaman "AI Kışı" adı verilen durgunluk dönemleri yaşamıştır.

- **1950'ler:** Alan Turing'in "Makineler düşünebilir mi?" sorusuyla başlayan Turing Testi dönemi.
- **1980'ler:** Uzman Sistemlerin yükselişi ve kural tabanlı yapay zekânın sınırlarına ulaşılması.
- **2010'lar:** Derin öğrenme devrimi, ekran kartlarının (GPU) paralel işlem gücünün yapay zeka eğitiminde kullanılmaya başlanması.
- **2020'ler:** Milyarlarca parametreye sahip Büyük Dil Modelleri (LLM) ve Üretken Yapay Zekâ çağı.

1.3. Makine Öğrenmesi (Machine Learning - ML)

Makine öğrenmesi, yapay zekânın verilerden öğrenmesini ve zamanla performansını artırmasını sağlayan bir alt dalıdır.

- **Geleneksel Programlama:** Girdi (Veri) + Kural (Kod) = Sonuç
- **Makine Öğrenmesi:** Girdi (Veri) + Sonuç (Etiket) = Kural

1.4. Derin Öğrenme (Deep Learning) ve Sinir Ağları

Derin öğrenme, insan beynindeki nöronların çalışma prensibinden ilham alan Yapay Sinir Ağları (Artificial Neural Networks) kullanan özel bir makine öğrenmesi türüdür. "Derin" kelimesi, ağıın içindeki girdi ve çıktı katmanları arasında bulunan "Gizli

Katmanların" (Hidden Layers) sayısının çok olmasını ifade eder. Ham veriyi alır ve kendisi anlamlı örüntüler çıkarır.

1.5. Yapay Zekânın Temel Yapıtaşları

- **Veri (Data):** Yapay zekânın yakıtıdır. Verinin kalitesi, modelin kalitesini doğrudan belirler.
- **Model:** Öğrenme süreci bittikten sonra ortaya çıkan matematiksel fonksiyondur.
- **Parametre (Parameter):** Modelin öğrenme aşamasında kendi kendine bulduğu ve güncellediği içsel ağırlıklar (weights) ve sapma (bias) değerleridir.
- **Eğitim (Training):** Modelin veriyi incelediği, hatasını fark edip parametrelerini güncellediği, devasa GPU kümelerinde gerçekleşen hesaplama yoğunluklu süreçtir.
- **Çıkarım (Inference):** Eğitim süreci bitmiş modelin gerçek dünyada kullanıma sunulmasıdır. Örneğin bir RAG sistemi çalıştırırken yapılan işlem "Çıkarım" işlemidir; model yeni bir şey öğrenmez, sadece bildiklerini kullanarak cevap üretir.

BÖLÜM 2: BÜYÜK DİL MODELLERİ (LLM) VE ÇALIŞMA MANTIKLARI

2.1. Büyük Dil Modeli (Large Language Model - LLM) Nedir?

Büyük Dil Modelleri (LLM), devasa miktarda metin verisiyle eğitilmiş, insan benzeri metinler anlayabilen ve üretebilen derin öğrenme algoritmalarıdır. Temel amaçları bir sonraki kelimeyi (token'ı) yüksek bir istatistiksel doğrulukla tahmin etmektir. ChatGPT, Claude ve senin Microsoft Foundry Local'da çalıştıracağın yerel modeller birer LLM'dir.

2.2. Transformer Mimarisi ve Attention (Dikkat) Mekanizması

2017 yılında yayınlanan "Attention Is All You Need" makalesi, günümüz LLM'lerinin temelini atan Transformer mimarisini tanıtmıştır. Eski sistemler cümleleri kelime kelime okurdu. Transformer ise **Attention (Dikkat)** mekanizması sayesinde cümlelerin tamamını aynı anda işler ve hangi kelimenin hangi kelimeyle ilişkili olduğuna "dikkat" eder.

Örneğin; "Banka kenarında oturan adam, banka hesabını kontrol etti" cümlesindeki iki farklı "banka" kelimesinin anlamını bu sayede ayırt eder.

2.3. Token ve Embedding (Vektörleştirme)

Modeller metinleri harf harf okumazlar. Metinler matematiksel dizilere dönüştürülür.

- **Token:** Metnin parçalanmış en küçük birimidir. Bir token bütün bir kelime, bir hece veya tek bir harf olabilir. Kabaca 100 token, yaklaşık 75 kelimeye denk gelir.
- **Embedding:** Kelimelerin veya token'ların çok boyutlu bir uzayda koordinatlara (vektörlere) dönüştürülmesidir. Bu sayede model kelimeler arasındaki anlamsal yakınlığı (örneğin "kral" ve "kraliçe") matematiksel olarak kavrar.

2.4. Context Window (Bağlam Penceresi)

Bir modelin tek bir seferde (bir prompt içinde) hatırlayabileceği ve işleyebileceği maksimum token sınırıdır. Bu sınır aşıldığında model sohbetin başını unutmaya başlar. RAG mimarisi, tüm dokümanı modele vermek yerine sadece soruyla ilgili parçayı bularak bu "bağlam penceresine" yerleştirdiği için hayat kurtarıcıdır.

2.5. Model Parametreleri: Temperature (Sıcaklık)

Modelin ürettiği cevapların ne kadar yaratıcı veya katı olacağını belirleyen ayardır. Genellikle 0.0 ile 1.0 arasında değer alır.

- **Temperature = 0.0:** Model tamamen deterministik (kesin) davranır, hep en yüksek olasılıklı kelimeyi seçer. RAG sistemlerinde, matematik hesaplamalarında ve kodlamada tercih edilir.
- **Temperature = 0.8 ve üzeri:** Model daha rastgele ve yaratıcı cevaplar üretir. Şiir veya hikaye yazımında kullanılır.

2.6. Hallucination (Halüsinasyon)

LLM'lerin bilmediği bir konu hakkında tamamen yanlış veya gerçekte var olmayan bir bilgiyi, son derece kendinden emin bir dille uydurmasıdır. RAG sistemleri, modeli doğrulanmış bir metne (PDF'e) sadık kalmaya zorladığı için halüsinasyon riskini minimuma indirir.

2.7. Prompt ve Prompt Engineering (İstem Mühendisliği)

- **Prompt:** Yapay zekaya verilen komut veya sorudur.
- **Prompt Engineering:** Modele en doğru ve verimli cevabı ürettirecek şekilde soruyu tasarlama sanatıdır. RAG mimarisinde sistem, kullanıcının sorusunu alır ve veritabanından bulduğu PDF metinleriyle birleştirerek modele arkada çok daha gelişmiş bir prompt sunar.

BÖLÜM 3: MODERN YAPAY ZEKÂ TEKNOLOJİLERİ

3.1. RAG (Retrieval-Augmented Generation)

RAG, büyük dil modellerinin (LLM) kendi eğitildikleri verilerle yetinmeyip, dışarıdan sağlanan güncel veya özel belgelere (örneğin senin hazırladığın bu PDF'e) bakarak cevap üretmesini sağlayan mimaridir. Modelin hafızasını taze tutar, kurum içi gizli verilerle çalışmasını sağlar ve modelin cevap uydurmasını (halüsinasyon) büyük oranda önler.

3.2. Vector Database (Vektör Veritabanı)

Geleneksel SQL veritabanları verileri satır ve sütunlar halinde tutarken, vektör veritabanları verileri çok boyutlu matematiksel uzaylarda "vektörler" (koordinatlar) olarak saklar. Bir RAG sisteminde PDF'in parçalanmış metinleri burada tutulur. Kullanıcı bir soru sorduğunda, sistem kelime eşleşmesi değil, "anlamsal yakınlık" araması yapar.

3.3. Fine-tuning (İnce Ayar)

Önceden eğitilmiş bir modelin, spesifik bir görevi daha iyi yapması için yeni bir veri setiyle yeniden eğitilmesidir. RAG modeli dışarıdan okumaya teşvik ederken, Fine-tuning modelin beynindeki matematiksel ağırlıkları kalıcı olarak değiştirir. RAG daha

ucuz ve esnektir, Fine-tuning ise modelin konuşma tarzını veya belirli bir uzmanlık alanındaki reflekslerini kalıcı olarak değiştirmek için kullanılır.

3.4. AI Agent (Yapay Zeka Ajanı)

Sadece soru cevaplayan değil, belirli bir hedefe ulaşmak için kendi kendine plan yapan, karar veren ve dış dünyadaki araçları kullanan sistemlerdir. Bir LLM sadece metin üretir; bir AI Agent ise internette arama yapar, kod yazar, bu kodu çalıştırıp hata alırsa kendi hatasını otonom olarak düzeltir.

3.5. Multi-Agent (Çoklu Ajan) Sistemleri

Birden fazla yapay zeka ajanının aynı projede farklı roller üstlenerek beraber çalışmasıdır. Örneğin bir yazılım projesinde bir ajan "Geliştirici", diğeri "Test Uzmanı", bir diğeri "Sistem Yöneticisi" rolünü üstlenip birbirleriyle otonom şekilde konuşarak bir mimariyi tamamlayabilirler.

3.6. Function Calling ve Tool Calling

Dil modelinin, dış dünyadaki API'leri veya fonksiyonları tetikleme yeteneğidir. Model, karmaşık bir matematik hesabı yapmak için arka planda bir hesap makinesi aracını (tool) veya sunucu durumunu kontrol etmek için bir API fonksiyonunu otonom olarak çağırabilir.

3.7. Memory (Hafıza)

Standart LLM sohbetleri, bağlam penceresi (context window) dolduğunda eskiyi unuttur. Modern yapay zeka sistemleri, kullanıcının geçmiş tercihlerini, yazdığı kodları veya projelerini kalıcı bir vektör veritabanında saklayarak uzun vadeli hafıza (Long-term Memory) oluşturur.

3.8. Multimodal AI (Çok Modlu Yapay Zeka)

Sadece metin değil; görsel, ses ve video gibi farklı veri formatlarını aynı anda anlayabilen ve üretebilen modellerdir. Gönderilen bir mimari çizimin içindeki hataları analiz edip sesli olarak yanıt vermek multimodal bir yetenektir.

Tamamdır, soruları tamamen metnin dışında bırakarak çok daha temiz ve profesyonel bir veri tabanı oluşturmaya devam ediyoruz. Bu sayede modelin kafası hiç karışmayacak.

Word dosyanı açıp, 3. Bölümün hemen altına aşağıdaki **BÖLÜM 4** metnini doğrudan kopyalayabilirsin. (Hazırladığımız bu bölüm, günümüzdeki modellerin avantajlarını ve kodlama kapasitelerini anlattığı için senin RAG sistemin için harika bir "karşılaştırma" veri seti olacak.)

BÖLÜM 4: GÜNÜMÜZDEKİ YAPAY ZEKÂ MODELLERİ

4.1. ChatGPT

- **Kim Geliştirdi:** OpenAI
- **Hangi Sürümleri Var:** GPT-3.5, GPT-4, GPT-4o, GPT-4o Mini
- **Avantajları:** En yaygın kullanılan modeldir. Akıl yürütme (reasoning) yeteneği çok güçlüdür. İnternet araması yapabilir ve gelişmiş veri analizi (Data Analysis) sunar.
- **Dezavantajları:** Kapalı kaynaklıdır (kodları gizlidir). Sürekli internet bağlantısı gerektirir ve kurumsal verilerle eğitilmesi maliyetlidir.
- **Ne İçin Kullanılır:** Genel asistanlık, metin yazarlığı, günlük problemlerin çözümü ve temel kodlama mantığını anlamak.
- **Kimler Kullanmalı:** Hızlı çözüm arayan son kullanıcılar ve genel amaçlı asistan arayan şirketler.
- **Gerçek Örnekler:** Bir projenin pazarlama metnini yazdırmak veya uzun bir e-posta zincirini özetlemek.

4.2. Gemini

- **Kim Geliştirdi:** Google
- **Hangi Sürümleri Var:** Nano (mobil için), Flash (hızlı), Pro (genel), Ultra (en güçlü). Modeller şu an 1.5 sürümündedir.
- **Avantajları:** İnanılmaz büyüklükte bir bağlam penceresine (Context Window) sahiptir; aynı anda saatlerce video veya binlerce sayfalık PDF okuyabilir. Google ekosistemiyle (Drive, Gmail, Docs) doğrudan entegredir.
- **Dezavantajları:** Bazen yaratıcılık yerine aşırı korumacı ve katı (güvenlik odaklı) cevaplar verebilir.

- **Ne İçin Kullanılır:** Çok büyük veri setlerinin tek seferde analizi ve Workspace üzerinden otomatik ofis işleri.
- **Kimler Kullanmalı:** Devasa log dosyaları veya yüzlerce sayfalık dökümanlar içinde kaybolan veri analistleri ve araştırmacılar.
- **Gerçek Örnekler:** Google Drive'daki 10 farklı araştırma raporunu tek bir prompt ile özetletip kıyaslamak.

4.3. Claude

- **Kim Geliştirdi:** Anthropic
- **Hangi Sürümleri Var:** Haiku (hızlı), Sonnet (dengeli ve zeki), Opus (ağır ve analitik). Güncel seri Claude 3.5'tir.
- **Avantajları:** Şu an sektörde kod yazma ve mantık kurma (özellikle Sonnet 3.5) konusunda en çok sevilen modeldir. İnsan gibi, çok doğal bir dil kullanır. "Artifacts" özelliği ile yazdığı kodu anında görsel bir arayüzde çalıştırıp gösterebilir.
- **Dezavantajları:** İnternet araması yapma veya diğer harici araçlarla entegrasyon konusunda bazen rakiplerinin gerisinde kalabilir.
- **Ne İçin Kullanılır:** İleri seviye yazılım geliştirme, mimari tasarım ve akademik metin çevirileri.
- **Kimler Kullanmalı:** Yazılım geliştiriciler (özellikle Python veya C gibi dillerde karmaşık mantık hatalarını çözenler) ve UI/UX tasarımcıları.
- **Gerçek Örnekler:** Karmaşık bir C kütüphanesindeki bellek sızıntısını (memory leak) tespit ettirmek ve düzelttirmek.

4.4. DeepSeek

- **Kim Geliştirdi:** DeepSeek (Çin merkezli bir yapay zeka şirketi)
- **Hangi Sürümleri Var:** DeepSeek-Coder, DeepSeek-V2, V3, R1
- **Avantajları:** Açık kaynak (open-source) dünyasında devrim yaratmıştır. Özellikle yüksek matematik gerektiren problemlerde ve algoritmik kod üretiminde inanılmaz başarılıdır. Maliyeti (API ücreti) batılı rakiplerine göre çok daha ucuzdur.
- **Dezavantajları:** Çince ve İngilizce dışındaki dillerde bazen yerel kültürel nüansları kaçırabilir.
- **Ne İçin Kullanılır:** Gelişmiş matematiksel akıl yürütme, optimizasyon problemleri ve otonom sistem kodlamaları.
- **Kimler Kullanmalı:** Makine öğrenmesi alanında çalışan mühendisler ve saf algoritmik performans arayan araştırmacılar.

- **Gerçek Örnekler:** Bir sensörden gelen gürültülü veriyi filtrelemek için kullanılacak matematiksel bir formülün optimizasyonu.

4.5. Llama

- **Kim Geliştirdi:** Meta (Facebook)
- **Hangi Sürümleri Var:** Llama 1, 2, 3 ve 3.1 serisi (8B, 70B, 400B gibi farklı parametre boyutlarında)
- **Avantajları:** Dünyanın en popüler "açık kaynaklı" (open-weights) dil modelidir. Geliştiriciler bu modeli alıp kendi bilgisayarlarında veya şirket sunucularında ücretsiz çalıştırabilirler.
- **Dezavantajları:** Doğrudan son kullanıcıya hitap eden hazır bir arayüzü (ChatGPT gibi) yoktur, geliştirici ortamı gerektirir.
- **Ne İçin Kullanılır:** Veri gizliliğinin kritik olduğu projeler, kurumsal RAG sistemleri ve yerel model barındırma.
- **Kimler Kullanmalı:** Kurum dışına (buluta) veri göndermenin yasak olduğu alanlarda çalışanlar ve sistem yöneticileri.
- **Gerçek Örnekler:** Dış internete kapalı bir bilgisayarda (örneğin Foundry Local üzerinde) yerel bir şirket içi asistan çalıştırmak.

4.6. Qwen

- **Kim Geliştirdi:** Alibaba Cloud
- **Hangi Sürümleri Var:** Qwen 1.5, 2, 2.5 (Farklı parametre boyutlarında)
- **Avantajları:** Açık kaynaklı modeller arasında Türkçe dil desteği ve başarısı en yüksek olan modellerden biridir. Kod yazma ve mantık yürütmede Llama serisine çok güçlü bir rakiptir.
- **Ne İçin Kullanılır:** Çok dilli (özellikle Asya dilleri ve Türkçe) yerel uygulamalar ve metin analizi.
- **Kimler Kullanmalı:** Yerel bir RAG sisteminde yüksek kalitede Türkçe çıktı almak isteyen geliştiriciler.

4.7. Mistral

- **Kim Geliştirdi:** Mistral AI (Fransa merkezli girişim)
- **Hangi Sürümleri Var:** Mistral 7B, Mixtral 8x7B (MoE mimarisi), Mistral Large
- **Avantajları:** Çok küçük boyutlarına rağmen (örneğin 7 Milyar parametre) kendisinden kat kat büyük modellerle yarışan bir performansa sahiptir. Çok hızlı çalışır ve donanım dostudur.

- **Ne İçin Kullanılır:** Düşük donanımlı cihazlarda (eski dizüstü bilgisayarlar veya edge cihazları) yapay zeka çalıştırmak.
- **Kimler Kullanmalı:** Kısıtlı RAM ve ekran kartına sahip olup, yine de kendi bilgisayarında yapay zeka denemeleri yapmak isteyenler.

Word dosyanı aç, 4. Bölümün bittiği yerden bir iki satır boşluk bırak ve bu harika kullanım senaryolarını anlatan 5. Bölüm'ü doğrudan yapıştır.

(İçindeki örnekleri senin mühendislik dünyana ve olası mülakat senaryolarına tam uyacak şekilde teknik detaylarla hazırladım, RAG modeli bu spesifik örnekleri okuduğunda çok daha zeki cevaplar verecektir.)

BÖLÜM 5: YAPAY ZEKÂYI GÜNLÜK HAYATTA VE İŞTE NASIL KULLANIRIZ?

Yapay zeka sadece teorik bir teknoloji değil, aynı zamanda günlük süreçleri hızlandıran çok güçlü bir asistandır. Bu asistanın verimi, kullanıcının mesleğine ve ihtiyacına göre şekillenir.

5.1. Öğrenciler ve Akademik Çalışmalar İçin

Öğrenciler yapay zekayı bir "cevap anahtarı" olarak değil, bir "özel ders hocası" olarak kullanmalıdır.

- **Karmaşık Kavramları Basitleştirme:** Anlaşılması zor bir mühendislik veya temel bilimler konusunu "Bunu 1. sınıf bilgisayar mühendisliği öğrencisine anlatır gibi anlat" komutuyla basitleştirebilir.
- **PDF ve Makale Özetleme:** Onlarca sayfalık akademik makaleleri veya ders notlarını sisteme yükleyip, "Bu belgedeki en kritik 5 formülü ve açıklamasını bana tablo olarak ver" şeklinde saatler sürecekle okumaları dakikalara indirebilir.
- **Yabancı Dil Pratiği:** Yapay zeka ile İngilizce mülakat simülasyonları yapılabilir veya yazılan İngilizce bir raporun gramer hataları düzeltilir.

5.2. Yazılımcılar ve Mühendisler İçin

Yapay zeka, modern yazılım geliştirme süreçlerinin ayrılmaz bir parçasıdır.

- **Kod Yazma ve Hata Ayıklama (Debugging):** Özellikle C ve Python gibi dillerde yazılan algoritmalarda karşılaşılan mantık hatalarını veya 'Segmentation Fault' gibi zorlayıcı durumları tespit edip çözüm üretebilir.
- **Proje Geliştirme:** Örneğin; donanım ve yazılımın bir arada çalıştığı bir İnsansız Kara Aracı (İKA) tasarlariken, otonom hareket için yazılacak sistem mimarisinin iskeletini oluşturabilir veya sensör verilerinin entegrasyonu için örnek Python scriptleri yazabilir.
- **Dokümantasyon:** Yazılan binlerce satırlık kodun ne işe yaradığını açıklayan "README" dosyalarını veya API dökümanlarını saniyeler içinde standartlara uygun şekilde hazırlar.

5.3. İçerik Üreticileri İçin

Yaratıcı süreçlerde yapay zeka, bir "fikir jeneratörü" olarak çalışır.

- **Metin ve Senaryo Üretimi:** YouTube videoları için dikkat çekici giriş metninden, Instagram gönderileri için etkileşim alacak açıklamalara (caption) kadar içerik iskeletleri oluşturur.
- **SEO ve Reklam:** Bir blog yazısının Google aramalarında üst sıralara çıkması için gerekli anahtar kelimeleri belirler ve hedef kitleye uygun reklam metinleri yazar.

5.4. İş Hayatı ve Profesyonel İletişim

Kurumsal hayatta yapay zeka, zaman yönetimi ve profesyonel imaj için kullanılır.

- **CV ve Ön Yazı (Cover Letter) Hazırlama:** Özellikle prestijli yaz dönemi online staj programlarına veya iş ilanlarına başvururken, adayın yeteneklerini ilandaki "aranan özellikler" ile eşleştirerek başvuru evraklarını optimize eder.
- **Resmi İletişim:** Kulüp etkinlikleri için sponsorluk arayışında şirketlere atılacak resmi e-postaları veya kurumlar arası toplantı taleplerini profesyonel bir üslupla kaleme alır.
- **Veri Analizi:** Excel tablolarındaki karmaşık finansal veya operasyonel verileri analiz edip, sunumlarda kullanılmak üzere yönetici özetleri çıkarır.

5.5. Günlük Hayat ve Kişisel Planlama

Zaman alıcı günlük kararları otomatize eder.

- **Seyahat Planlaması:** "Bana düşük bütçeli, 3 günlük kültürel bir İstanbul gezi rotası çıkar ve saat saat planla" gibi özel komutlarla seyahat acentesi gibi çalışır.

- **Alışveriş ve Beslenme:** Evdeki malzemeleri yazarak ("Dolapta tavuk, krema ve makarna var") yapılabilecek yemek tariflerini veya kişisel kalori hedefine uygun haftalık alışveriş listelerini oluşturur.

(Kopyalaman gereken metin burada bitti)

Aşağıdaki soruları Word dosyana **ekleme**, sadece kendi RAG testlerini yapacağın zaman kullanmak üzere kenardaki not defterine kopyala:

BÖLÜM 6: HANGİ YAPAY ZEKÂYI NE İÇİN KULLANMALIYIM?

Yapay zeka dünyasında "en iyi" tek bir model yoktur; "göreve en uygun" model vardır. Doğru aracı seçmek, elde edilecek sonucun kalitesini ve işlem süresini doğrudan etkiler. Aşağıdaki tablo, farklı ihtiyaç senaryolarında hangi modelin neden tercih edilmesi gerektiğini özetlemektedir.

6.1. Kullanım Senaryoları ve Model Eşleştirmeleri

İhtiyaç / Görev	En Uygun Araçlar	Neden Bu Araç Seçilmeli?
İleri Seviye Kod Yazma ve Hata Ayıklama	Claude 3.5 Sonnet, DeepSeek-Coder	Kod mantığını insan gibi anlama, karmaşık mimari kurma ve hataları (bug) bulmada sektör lideridirler. Claude görsel arayüz sunarken, DeepSeek algoritmik güç sağlar.
Uzun Belge ve PDF Analizi (RAG)	Gemini 1.5 Pro, Claude 3.5	Devasa bağlam pencereleri (Context Window) sayesinde yüzlerce sayfalık raporları tek seferde okuyup içinden spesifik verileri çıkarabilirler.
Google Ekosistemi ile Entegre Çalışma	Gemini	Gmail, Google Docs ve Google Drive içindeki verilerle doğrudan konuşabilme ve otomasyon sağlama yeteneği vardır.

Yüksek Matematik ve Algoritma Çözümleri	DeepSeek	Matematiksel akıl yürütme ve optimizasyon problemlerini çözmek üzere özel olarak eğitilmiş açık kaynaklı bir mimariye sahiptir.
Tamamen Yerel (Çevrimdışı) Çalışma	Llama 3, Qwen, Mistral	İnternet bağlantısı gerektirmeden şirket bilgisayarlarında veya Foundry Local gibi sistemlerde veri gizliliği sağlayarak çalıştırılabilirler.
Genel Asistanlık ve İnternet Araması	ChatGPT (GPT-4o)	Gündelik sorular, hızlı internet araştırmaları, veri analizi ve genel metin üretimi için en dengeli ve kapsamlı modeldir.
Düşük Donanımlı Cihazlarda Kullanım	Mistral 7B	Küçük parametre boyutuna rağmen yüksek performans göstererek eski dizüstü bilgisayarlarda bile akıcı çalışır.

6.2. Karar Verme Ağacı: Nasıl Seçim Yapmalıyım?

Araç seçerken şu üç soruyu kendinize sorun:

- Verim gizli mi kalmalı?** Evet ise -> Yerel Modeller (Llama, Qwen). Hayır ise -> Bulut Modeller (ChatGPT, Claude).
- İşlemin türü nedir?** Kod ise -> Claude/DeepSeek. Uzun okuma ise -> Gemini/Claude.
- İnternete ihtiyaç var mı?** Güncel veri lazımsa -> ChatGPT/Perplexity. Kavramsal analiz ise -> Yerel modeller.

BÖLÜM 7: 2026 YILINDA ÖĞRENMEN GEREKEN TEKNOLOJİLER

Bir yapay zeka sistemini sıfırdan kurmak veya donanım projelerine (otonom araçlar gibi) entegre etmek için sadece modelleri bilmek yetmez. Bu modelleri çevreleyen mühendislik ekosistemine de hakim olmak gerekir.

7.1. Temel Programlama ve İşletim Sistemleri

- **Python:** Yapay zekanın tartışmasız ortak dilidir. Modelleri eğitmek, veri analizi yapmak ve API'leri kullanmak için bir numaralı araçtır.
- **C / C++:** Yapay zeka modellerinin ürettiği kararları donanıma dökmek (örneğin bir robotun motor sürücülerine komut göndermek) ve gömülü sistemlerde yüksek performans elde etmek için vazgeçilmezdir.
- **Linux (Ubuntu):** Çoğu yapay zeka sunucusu ve yerel model barındırma aracı Linux üzerinde çok daha stabil ve performanslı çalışır. Temel terminal komutlarına hakimiyet şarttır.

7.2. Geliştirme ve Dağıtım Araçları (DevOps)

- **Git ve GitHub:** Yazılan kodların versiyonlarını tutmak ve özellikle mekanik, elektronik ve yazılım bileşenlerinden oluşan çok disiplinli takımlarda kodu çakışmadan birleştirmek için kullanılır.
- **Docker:** Geliştirdiğin bir RAG sisteminin veya AI uygulamasının senin bilgisayarında çalışıp başkasında hata vermemesi için projeyi tüm bağımlılıklarıyla "konteyner" içine hapseden sistemdir.
- **FastAPI:** Geliştirdiğin yapay zeka modelini dış dünyanın (veya takım arkadaşlarının yazdığı arayüzlerin) kullanımına açmak için Python tabanlı, çok hızlı bir web API iskeletidir.

7.3. Modern AI ve RAG Çerçeveleri (Frameworks)

- **LangChain & LlamaIndex:** LLM'leri (Büyük Dil Modellerini) diğer veri kaynaklarıyla, vektör veritabanlarıyla ve dış araçlarla bağlamak için kullanılan "orquestrasyon" kütüphaneleridir. Kod yazımını inanılmaz hızlandırırlar.
- **Vector Database (ChromaDB / Pinecone):** Metinlerin vektörleştirilmiş (embedding) hallerini saklayan ve RAG mimarisinin arama motoru gibi çalışan özel veritabanlarıdır.

- **Ollama / Microsoft Foundry Local:** Büyük modelleri (Llama, Mistral) buluta göndermeden, tamamen kendi bilgisayarının RAM ve GPU gücüyle yerel olarak çalıştırmanı sağlayan platformlardır.
- **Hugging Face:** Yapay zeka dünyasının GitHub'ıdır. Açık kaynaklı tüm modeller, veri setleri ve tokenizer'lar buradan indirilir.

BÖLÜM 8: GERÇEK PROJE FİKİRLERİ VE KULLANIM SENARYOLARI

Yapay zeka araçlarını ve RAG mimarisini teorik olarak bilmek mülakatlarda bir yere kadar etkilidir. Asıl fark yaratan, bu teknolojileri spesifik bir endüstri problemine çözüm üreten somut projelere dönüştürmektir.

8.1. Hukuk Asistanı (Legal RAG)

Hukuk bürolarının en büyük sorunu, binlerce sayfalık geçmiş mahkeme kararları ve içtihatlar arasında emsal dava aramaktır. RAG mimarisi ile kurumun tüm geçmiş dava dosyaları vektör veritabanına aktarılır. Avukatlar sisteme "2020 yılındaki ticari sözleşme feshi davalarındaki tazminat oranlarını getir" şeklinde sorular sorarak günlerce sürecektir araştırmayı saniyelere indirebilir.

8.2. Sağlık Araştırma Sistemi (Medical RAG)

Tıp araştırmacılarının güncel makaleleri ve klinik deney sonuçlarını takip etmesi için tasarlanır. PubMed gibi veritabanlarındaki spesifik hastalık raporları sisteme yüklenir. Model, doktorlara ilaç etkileşimleri veya yeni tedavi protokolleri hakkında halüsinasyon görmeden, doğrudan yüklenen makalelere (PDF'lere) atıf yaparak güvenilir özetler sunar.

8.3. Kurumsal İK ve CV Analiz Asistanı

Büyük şirketlerin işe alım süreçlerini otomatize eden ajan (AI Agent) sistemidir. İnsan kaynakları departmanına gelen binlerce CV, RAG mimarisi üzerinden okunur. Sistem sadece kelime eşleştirmesi yapmaz; adayın deneyimlerini, iş ilanındaki karmaşık teknik gereksinimlerle anlamsal olarak analiz edip en uygun adayları kısa listeye alır.

8.4. Savunma Sanayi ve Teknik Şartname Analizörü

Mühendislik projelerinde, özellikle savunma sanayisinde uyulması gereken binlerce sayfalık ISO standartları ve askeri teknik şartnameler (MIL-STD) bulunur. Mühendisler tasarım yaparken "Bu motorun çalışma sıcaklık aralığı standartlara uygun mu?" diye sorduğunda, asistan doğrudan şartnamenin ilgili maddesini bulup ekrana getirir ve kural ihlallerini önler.

8.5. E-Ticaret ve Müşteri Hizmetleri Chatbotu

Klasik kural tabanlı (if-else) sohbet botlarının aksine, şirketin güncel kargo prosedürlerini, iade şartlarını ve ürün stoklarını sürekli vektör veritabanından okuyan dinamik sistemlerdir. Müşterinin spesifik sorununu anlayıp, şirketin güncel politikalarına göre insansı ve çözüm odaklı yanıtlar üretirler.

BÖLÜM 9: YAPAY ZEKÂ TERİMLERİ SÖZLÜĞÜ

Yapay zeka ekosisteminde okuduğunuz teknik dokümanları ve makaleleri tam olarak kavrayabilmek için aşağıdaki temel terminolojiye hakim olmak gereklidir.

- **Agent (Ajan):** Belirli bir amaca ulaşmak için otonom olarak düşünebilen, plan yapabilen ve çeşitli araçları (web araması, kod çalıştırma, API çağırma vb.) kullanabilen yapay zeka sistemi.
- **API (Application Programming Interface):** Farklı yazılımların veya platformların birbiriyle iletişim kurmasını sağlayan köprü. Bir yapay zeka modelini başka bir koda veya uygulamaya entegre etmek için kullanılır.
- **Attention (Dikkat Mekanizması):** Transformer mimarisinde, modelin bir cümledeki kelimeler arasındaki anlamsal ilişkileri ve önem derecelerini aynı anda tartmasını sağlayan matematiksel mekanizma.
- **Context Window (Bağlam Penceresi):** Modelin tek bir işlemde (prompt) hatırlayabileceği ve işleyebileceği maksimum veri kapasitesidir. Genellikle "token" cinsinden ölçülür.

- **Embedding (Vektörleştirme):** Metinlerin, resimlerin veya seslerin makine tarafından anlaşılabilmesi için çok boyutlu matematiksel sayılara (vektörlere) dönüştürülmesi işlemi.
- **Fine-tuning (İnce Ayar):** Önceden büyük verilerle eğitilmiş bir temel modelin, belirli bir görevi (örneğin sadece hukuk diliyle konuşmayı) daha iyi yapabilmesi için yeni ve spesifik bir veri setiyle yeniden eğitilmesi.
- **GPU (Grafik İşlem Birimi):** Yapay zeka modellerinin eğitimi ve çıkarımı (inference) için gereken devasa, eşzamanlı ve paralel matematiksel hesaplamaları yapan donanım.
- **Hallucination (Halüsinasyon):** Dil modelinin gerçekte var olmayan, yanlış veya uydurma bir bilgiyi son derece mantıklı ve kendinden emin bir dille doğruymuş gibi sunması.
- **Inference (Çıkarım):** Eğitimi tamamlanmış bir modelin, kendisine verilen yeni bir komuta yanıt üretmesi (çalışma anı) işlemidir.
- **LLM (Large Language Model - Büyük Dil Modeli):** Milyarlarca parametreden oluşan ve insan dilini anlama, üretme ve mantık kurma yeteneğine sahip devasa sinir ağları mimarisi (Örn: GPT-4, Llama 3).
- **LoRA (Low-Rank Adaptation):** Devasa bir modeli sıfırdan eğitmek yerine, sadece parametrelerinin çok küçük bir kısmını güncelleyerek çok daha az donanım gücüyle ve ucuz bir şekilde fine-tuning yapmayı sağlayan teknik.
- **MCP (Model Context Protocol):** Yapay zeka modellerinin dış dünyaya, yerel veri kaynaklarına veya veritabanlarına standart ve güvenli bir yoldan bağlanmasını sağlayan açık kaynaklı protokol.
- **Prompt (İstem):** Yapay zeka modeline ne yapması gerektiğini, hangi formatta cevap vermesi gerektiğini söyleyen kullanıcı girdisi veya komut.
- **Quantization (Kuantizasyon):** Bir modelin matematiksel hassasiyetini (örneğin kayan noktalı sayıları 16-bit'ten 4-bit'e) düşürerek, modelin zekasından çok az ödün verip dosya boyutunu küçültme ve standart bilgisayarlarda çalışmasını sağlama işlemi.
- **RAG (Retrieval-Augmented Generation):** Dil modelinin, cevabı kendi hafızasından uydurmak yerine dışarıdan (vektör veritabanından) çağrılan doğrulanmış metinleri okuyarak cevap üretmesini sağlayan mimari.
- **Token:** Yapay zekanın metni okurken böldüğü en küçük anlam birimi. Bir token bazen bir kelimeye, bazen bir heceye, bazen de tek bir harfe denk gelebilir.
- **Training (Eğitim):** Modelin devasa boyuttaki ham veriyi inceleyerek içindeki dil bilgisi kurallarını ve örüntüleri öğrendiği, milyonlarca dolar maliyeti olan yoğun hesaplama süreci.
- **Transformer:** 2017'de yayınlanan, metni kelime kelime sırayla okumak yerine cümlelerin tamamını aynı anda işleyen ve günümüz üretken yapay zekasının (Generative AI) temelini oluşturan derin öğrenme mimarisi.

- **Vector Database (Vektör Veritabanı):** Kelimelerin veya dokümanların matematiksel karşılıklarını (embedding) saklayan ve klasik arama yerine "anlamsal yakınlık" araması yapan özel veritabanı türü.

BÖLÜM 10: YAPAY ZEKÂ İLE VERİMLİ ÇALIŞMA REHBERİ

Yapay zeka modellerinin performansı, kullanıcının ona verdiği komutun (prompt) kalitesiyle doğrudan orantılıdır. Modeli bir "sihirli değnek" olarak değil, yetenekli ama yönlendirilmeye muhtaç bir stajyer olarak görmek en doğru yaklaşımdır.

10.1. Yapay Zekâya Nasıl Doğru Soru Sorulur? (Rol ve Bağlam)

Etkili bir prompt yazmak için şu üç bileşen mutlaka komutun içinde olmalıdır:

1. **Rol Verin:** Modele kim olduğunu söyleyin. (Örn: "Sen kıdemli bir sistem mimarisin" veya "Sen bir veri analistisin.")
2. **Bağlam Sağlayın:** İşin arka planını açıklayın. Model sorunun nerede kullanılacağını bilmelidir.
3. **Format Belirleyin:** Çıktının nasıl görünmesini istediğinizi netleştirin (Örn: "Maddeler halinde yaz", "Tablo olarak ver" veya "Sadece kod bloğu olarak döndür").

10.2. Kötü Prompt ile İyi Prompt Arasındaki Fark

Aşağıdaki örnek, komutun detaylandırıldığında sonucun nasıl değiştiğini açıkça göstermektedir.

- **Kötü Prompt:** "Bana bir web sitesi kodu yaz."

- *Neden Kötü?* Çok genel. Modelin hangi dili kullanacağını, sitenin amacını veya tasarımını belirtmiyor. Standart ve muhtemelen işe yaramayan bir şablon üretecektir.
- **İyi Prompt:** "Sen kıdemli bir Frontend geliştiricisin. React ve Tailwind CSS kullanarak, karanlık (dark) temaya sahip duyarlı (responsive) bir e-ticaret ürün listeleme sayfası yazmanı istiyorum. Kodu yeni başlayanların anlayabileceği şekilde, her fonksiyonun yanına yorum satırları ekleyerek tek bir dosya halinde ver."
- *Neden İyi?* Rol belli (Frontend geliştirici), kullanılacak teknolojiler belli (React, Tailwind), amaç belli (e-ticaret sayfası), tasarım detayı belli (karanlık tema) ve çıktı formatı (yorum satırlı tek dosya) net olarak tanımlanmış.

10.3. Bilgiyi Doğrulama ve Halüsinasyon Kontrolü

Yapay zeka ne kadar zeki görünürse görünsün, istatistiksel bir tahmin motorudur. Kesin gerçekler beklediğinizde şu doğrulama adımlarını izleyin:

- **Çapraz Sorgulama:** Modelin verdiği spesifik bir veriyi (örneğin bir kütüphanenin sürüm notları) doğrudan arama motorlarında veya resmi dökümanlarda teyit edin.
- **Kaynak İsteyin:** Modele, "Bu bilgiyi hangi akademik makaleye veya resmi dokümana dayandırarak verdin?" diye sorun. RAG kullanmıyorsanız, bazen uydurma kaynaklar (halüsinasyon) sunabileceğini unutmayın.

10.4. İnsan Kontrolü (Human-in-the-Loop)

Yapay zeka hiçbir zaman sürecin tek karar vericisi olmamalıdır.

- **Tamamen Güvenilebilecek İşler:** Metin özetleme, beyin fırtınası yapma, kod mantığını açıklama, çeviri yapma ve taslak metin oluşturma.
- **İnsan Kontrolü Gerektiren İşler:** Doğrudan canlıya alınacak (production) yazılımlar, yasal bağlayıcılığı olan sözleşmeler, tıbbi kararlar ve finansal analizler mutlaka bir uzman tarafından gözden geçirilmelidir.

BÖLÜM 11: MULTİMODAL YAPAY ZEKÂ (GÖRSEL, SES VE VIDEO ÜRETİMİ)

Şu ana kadar genellikle metin tabanlı Büyük Dil Modellerini (LLM) inceledik. Ancak yapay zekânın gerçek devrimi, farklı veri türlerini (metin, görsel, ses, video) aynı anda anlayabilen ve üretebilen "Multimodal" (Çok Modlu) sistemlerle yaşanmaktadır. Bu sistemler, modern içerik üretimini ve tasarım süreçlerini kökten değiştirmektedir.

11.1. Multimodal Yapay Zekâ Nedir?

Geleneksel modeller tek bir veri tipine odaklanırken (sadece metin okuma veya sadece fotoğraf tanıma), multimodal modeller farklı veri tiplerini aynı matematiksel uzayda eşleştirerek işler. Örneğin, bir metin komutu girildiğinde ("Neon ışıklı bir sokakta yürüyen robot"), model bu kelimelerin görsel dünyadaki piksel karşılıklarını hesaplayarak sıfırdan bir resim veya video üretebilir.

11.2. Görsel Üretim Modelleri ve Farkları

Görsel üreten yapay zekâlar genellikle "Diffusion" (Difüzyon) adı verilen bir mimari kullanır. Bu sistemler, tamamen parazitli (kumlu) bir görseli adım adım netleştirerek (denoising) hedeflenen görüntüyü oluşturur.

- **Midjourney:** Sektördeki en yüksek sanatsal ve fotogerçekçi kaliteye sahip modeldir. Mimari görselleştirmeler, karakter tasarımları ve konsept sanat üretiminde endüstri standardı kabul edilir. Discord üzerinden veya kendi web arayüzünden kullanılır.
- **DALL-E 3:** OpenAI tarafından geliştirilmiştir ve ChatGPT içine entegredir. Midjourney kadar sanatsal olmasa da, verilen karmaşık metin komutlarını harfiyen anlama ve görselin içine istenilen metinleri hatasız yazabilme konusunda çok başarılıdır.
- **Stable Diffusion:** Stability AI tarafından açık kaynak (open-source) olarak sunulmuştur. Tamamen ücretsizdir ve kişisel bilgisayarlarda (yeterli donanımla) çevrimdışı olarak çalıştırılabilir. Geliştiriciler tarafından kendi özel verileriyle kolayca eğitilebilir.

11.3. Video Üretim Teknolojileri

Yapay zekâ ile video üretimi, her bir karenin (frame) birbiriyle tutarlı olmasını ve fizik kurallarının korunmasını gerektirdiği için en fazla işlem gücü isteyen alandır.

- **Sora:** OpenAI tarafından geliştirilen, tek bir metin komutuyla 60 saniyeye kadar yüksek çözünürlüklü, fizik kurallarına uygun ve kamera hareketleri tutarlı videolar üretebilen güçlü bir modeldir.
- **Runway (Gen Serisi):** Metinden videoya veya statik bir görselden videoya hareket sağlayan, özellikle film endüstrisi, kurgucular ve reklamcılar tarafından aktif olarak kullanılan profesyonel bir araçtır.
- **Kling ve Luma Dream Machine:** Karakterlerin yüz ifadelerini, sıvı akışkanlığını ve karmaşık kamera açılarını saniyeler içinde simüle edebilen popüler video üretim motorlarıdır.

11.4. Ses, Konuşma ve Müzik Üretimi

Metinleri doğal insan sesine dönüştürme (Text-to-Speech) ve sıfırdan müzik besteleme alanında çalışan yapay zekâ modelleridir.

- **ElevenLabs:** Metni sese çevirme konusunda sektörün en gerçekçi modelidir. Sesteki duyguyu, vurguyu ve hatta nefes alışları bile insan doğallığında kopyalayabilir. Aynı zamanda var olan bir sesi klonlama (Voice Cloning) yeteneği çok yüksektir.
- **Suno AI ve Udio:** Sadece yazılı komutlar girerek (örneğin "80'ler tarzında, uzay yolculuğunu anlatan bir elektronik pop şarkısı") sıfırdan vokalleri, enstrümanları ve melodisi olan tam prodüksiyonlu müzik parçaları üreten platformlardır.

11.5. Kullanım Riskleri ve Sınırlandırmalar

- **Telif Hakkı (Copyright):** Multimodal modellerin ürettiği görsellerin veya müziklerin ticari kullanımı, eğitim verilerinin telifli olup olmadığına dair tartışmalar nedeniyle yasal bir gri alan oluşturmaktadır.
- **Deepfake ve Dezenformasyon:** Ses klonlama araçları ve gerçekçi video motorları, siber güvenlik ve kimlik sahtekarlığı açısından ciddi riskler barındırdığı için platformlar tarafından sıkı güvenlik kilitleriyle korunmaya çalışılmaktadır.

BÖLÜM 12: İLERİ SEVİYE PROMPT MÜHENDİSLİĞİ TEKNİKLERİ

Yapay zekâ modellerine sadece "soru sormak" ile onlara "mühendislik yapmak" arasındaki fark, kullanılan komut (prompt) tekniklerinde gizlidir. Karmaşık algoritmik problemleri, veri analizlerini veya mantıksal çıkarımları çözerken modelin hata yapmasını engellemek için akademik olarak kanıtlanmış komut mimarileri kullanılır.

12.1. Zero-Shot Prompting (Sıfır Örnekli İstem)

Modelden, kendisine daha önce hiçbir format örneği verilmeden doğrudan bir görevi yapmasının istendiği en temel kullanım şeklidir. Model, sadece devasa eğitim verisine güvenerek cevap üretir.

- **Kurumsal Örnek:** "Aşağıdaki sunucu hata logunu (server error log) incele ve hatanın aciliyet durumunu sadece [Kritik / Uyarı / Bilgi] etiketlerinden biriyle sınıflandır: *[Error 502: Bad Gateway at 10.0.0.4]*"
- **Kullanım Alanı:** Basit sınıflandırma işlemleri, metin çevirileri ve doğrudan bilgi sorguları gibi modelin zaten çok iyi bildiği standart görevler için uygundur.

12.2. Few-Shot Prompting (Az Örnekli İstem)

Büyük dil modelleri bazen istenen spesifik formatı (örneğin JSON veya XML çıktıları) tek seferde anlayamaz. Few-Shot tekniğinde, asıl soruyu sormadan önce modele "Soru-Cevap" formatında birkaç tane doğru çözülmüş, standartlara uygun örnek sunulur. Model bu şablonu (pattern) kopyalayarak asıl veriyi hatasız işler.

- **Kurumsal Örnek:**
 - *Girdi:* "Ahmet Yılmaz, Yazılım Mühendisi, ahmet@sirket.com" -> *Çıktı:* {"isim": "Ahmet Yılmaz", "rol": "Yazılım Mühendisi", "email": "ahmet@sirket.com"}
 - *Girdi:* "Ayşe Kaya, Veri Analisti, ayse.k@sirket.com" -> *Çıktı:* {"isim": "Ayşe Kaya", "rol": "Veri Analisti", "email": "ayse.k@sirket.com"}
 - *Girdi:* "Can Demir, Sistem Yöneticisi, can@sirket.com" -> *Çıktı:* [Modelden beklenen JSON formatlı yanıt]
- **Kullanım Alanı:** Düzensiz verileri (unstructured data) temizleyip veritabanına yazılacak standart bir formata çevirme işlemleri.

12.3. Chain of Thought - CoT (Düşünce Zinciri)

Özellikle algoritmik karmaşıklığı yüksek olan veya matematiksel hesaplama gerektiren problemlerde yapay zekânın çuvallamasını engelleyen en güçlü tekniktir. Dil modelleri kelimeleri ardışık tahmin ederek çalıştığı için, doğrudan sonuca atlamaya çalıştıklarında hesap hatası yaparlar. Modele "*Adım adım düşün*" komutu verilerek her bir işlem basamağını yazması sağlanır.

- **Kötü Yaklaşım:** "Sistemde 3 veritabanı sunucusu var. Birinci sunucu saniyede 500, ikincisi 300 sorgu kaldırıyor. Saniyede 1200 sorgu geliyorsa ve üçüncü sunucunun kapasitesi 400 ise sistem çöker mi? Sonucu yaz." (Model hesaplamayı atlayıp yanlış tahmin yapabilir).
- **CoT Yaklaşımı:** "Sistemde 3 veritabanı sunucusu var... (soru aynı). Lütfen sonucu vermeden önce toplam kapasiteyi ve gelen yükü adım adım hesaplayarak yaz."
- **Sonuç:** Model her adımı bir alt satıra yazarak kapasiteleri toplar ($500+300+400=1200$), gelen yükü kıyaslar ve nihai kararı hata yapmadan verir. Hata oranı %80'e varan oranlarda düşer.

12.4. Tree of Thoughts - ToT (Düşünce Ağacı)

CoT (Düşünce Zinciri) tekniğinin daha da geliştirilmiş halidir. Modelin tek bir mantık çizgisi üzerinden gitmesi yerine, problemi farklı dallara ayırması, her bir dalın (alternatif çözümün) olası sonuçlarını kendi kendine değerlendirmesi istenir.

- **Kurumsal Örnek:** "Yeni kurulacak bir e-ticaret platformunun arka uç (backend) mimarisini tasarlıyoruz. Lütfen 'Mikroservis (Microservices) Mimari' ve 'Monolitik (Monolithic) Mimari' olmak üzere iki farklı düşünce dalı (tree) oluşturun. Her iki dal için maliyet, ölçeklenebilirlik ve bakım zorluğunu adım adım değerlendir. En son ikisini kıyaslayıp optimal olanı seç."
- **Kullanım Alanı:** Yazılım mimarisi tasarlamak, DevOps altyapısı kurmak veya çok değişkenli mühendislik optimizasyon problemleri çözmek.

12.5. Self-Consistency (Kendi İçinde Tutarlılık)

Aynı karmaşık problemin sisteme (veya API üzerinden farklı bağlantılarla) birden fazla kez sorulması ve modelin ürettiği farklı çözüm yolları arasında en çok aynı sonuca ulaşan (çoğunluk oyu alan) cevabın doğru kabul edilmesi yöntemidir. Özellikle kritik hata ayıklama (debugging) süreçlerinde modelin sağlamasını yapmak için kullanılır.

BÖLÜM 13: YAPAY ZEKÂDA ETİK, GÜVENLİK VE HUKUKİ PROBLEMLER

Yapay zeka sistemleri şirketlere devasa bir hız kazandırırken, doğru yönetilmediklerinde kurumsal itibar kaybına, devasa veri sızıntılarına ve hukuki davalara yol açabilirler. Bir mühendisin veya yöneticinin, AI sistemlerini kurarken bu riskleri bilmesi ve önlem alması zorunludur.

13.1. Veri Gizliliği (Data Privacy) ve Kurumsal Sızıntılar

Halka açık büyük dil modelleri (ChatGPT, Claude vb.), kullanıcıların girdikleri verileri (eğer özel olarak kapatılmadıysa) kendilerini eğitmek için kullanabilir. Bu durum, şirketlerin en gizli verilerinin dışarı sızmasına neden olabilir.

- Standart Endüstri Örneği:** 2023 yılında büyük bir teknoloji şirketinin (Samsung) mühendisleri, kapalı kaynak kodlarındaki bir hatayı bulması için ChatGPT'ye şirket içi gizli kodları kopyalayıp yapıştırmıştır. Bu durum, firmanın gizli çip mimarisi verilerinin OpenAI sunucularına geçmesine ve halka açık modelin eğitim setine sızmasına yol açmıştır.

13.2. Model Önyargıları (AI Bias) ve Ayrımcılık

Yapay zeka modelleri, internetteki devasa verilerle eğitilir. Eğer eğitim verisinde tarihsel bir eşitsizlik veya insan önyargısı varsa, model bunu "matematiksel bir doğru" olarak kabul edip kararlarına yansıtır.

- Standart Endüstri Örneği:** Küresel bir e-ticaret devi (Amazon), işe alım süreçlerini hızlandırmak için geçmiş 10 yıldaki CV'leri tarayarak bir yapay zeka aracı geliştirmiştir. Ancak geçmiş 10 yılda teknoloji sektöründe ağırlıklı olarak erkekler çalıştığı için, model "kadın" kelimesi geçen CV'leri otomatik olarak eleme eğilimi göstermiş ve proje iptal edilmek zorunda kalmıştır.

13.3. Halüsinasyonun Yasal ve Hayati Riskleri

Modellerin bilgiyi uydurma (halüsinasyon) eğilimi, özellikle hukuk ve tıp gibi "sıfır hata" gerektiren alanlarda insan kontrolü olmadan kullanıldığında felaketlere yol açabilir.

- **Standart Endüstri Örneği:** Amerika Birleşik Devletleri'nde bir avukat, savunma dilekçesini hazırlarken ChatGPT kullanmış ve modelin kendisine sunduğu "emsal mahkeme kararlarını" doğrudan hakime sunmuştur. Ancak modelin verdiği mahkeme isimlerinin, dosya numaralarının ve kararların tamamen uydurma olduğu ortaya çıkmış, avukatın meslek lisansı tehlikeye girmiştir.

13.4. Telif Hakları (Copyright) İhlalleri

Görsel ve metin üreten yapay zeka modellerinin "sıfırdan yaratım" yapabilmesi için telif hakkı olan milyonlarca kitap, makale ve sanat eseriyle izinsiz eğitildiği iddia edilmektedir.

- **Standart Endüstri Örneği:** The New York Times gazetesi, kendi yazdıkları milyonlarca makalenin ChatGPT'nin eğitim setinde izinsiz kullanıldığını ve modelin bazen gazeteye ait ücretli makaleleri kullanıcılara ücretsiz olarak aynen kopyaladığını iddia ederek milyarlarca dolarlık telif hakkı ihlali davası açmıştır.

13.5. Kötüye Kullanım: Deepfake ve Dezenformasyon

Özellikle çok modlu (multimodal) sistemlerin gelişmesiyle, sahte ses ve video (Deepfake) üretimi siber güvenliğin en büyük tehdidi haline gelmiştir. Bu durum kimlik hırsızlığına ve toplumsal manipülasyonlara zemin hazırlamaktadır.

- **Standart Endüstri Örneği:** Hong Kong'daki çok uluslu bir şirketin finans çalışanı, yapay zeka ile oluşturulmuş "Deepfake" bir video konferans görüşmesine katılmıştır. Ekranda şirketinin CFO'sunu ve diğer iş arkadaşlarını birebir aynı ses ve görüntüyle gören çalışan, dolandırıcıların talimatlarına uyarak şirketin 25 milyon dolarını sahte hesaplara transfer etmiştir.

BÖLÜM 14: PYTHON İLE SIFIRDAN RAG SİSTEMİ KURULUMU

Bir RAG sistemi, temelde üç ana bileşenden oluşur: **Doküman Yükleyici**, **Vektör Veritabanı** ve **LLM (Büyük Dil Modeli) Bağlayıcısı**. Bu örnekte, endüstride standart olan LangChain kütüphanesini ve vektör veritabanı olarak ChromaDB kullanacağız.

14.1. Gerekli Kütüphanelerin Kurulumu

Öncelikle terminal veya komut satırı üzerinden gerekli paketleri kurmalıyız:

Bash

```
pip install langchain langchain-community langchain-chroma chromadb  
langchain-openai
```

14.2. Adım Adım RAG Uygulaması (Python Kodu)

Aşağıdaki kod, elindeki PDF dokümanlarını okuyup onları parçalara ayıran, vektörlere dönüştüren ve sisteme soru sorulabilir hale getiren temel iskelettir:

Python

```
import os  
from langchain_community.document_loaders import PyPDFLoader  
from langchain_text_splitters import RecursiveCharacterTextSplitter  
from langchain_openai import OpenAIEmbeddings, ChatOpenAI  
from langchain_chroma import Chroma  
from langchain.chains import RetrievalQA
```

1. DOKÜMAN YÜKLEME VE PARÇALAMA

PDF dosyamızı yüklüyoruz

loader = PyPDFLoader("teknik_rapor.pdf")

data = loader.load()

Metni, modelin kolay işleyebileceği küçük parçalara (chunk)
ayırıyoruz

text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000,
chunk_overlap=100)

chunks = text_splitter.split_documents(data)

2. VEKTÖR VERİTABANI OLUŞTURMA

Parçalanmış metinleri sayısal vektörlere (embedding) dönüştürüp
ChromaDB'ye kaydediyoruz

```
vector_db = Chroma.from_documents(  
    documents=chunks,  
    embedding=OpenAIEmbeddings()  
)
```

3. RAG ZİNCİRİNİ KURMA (RETRIEVAL)

Kullanıcı soru sorduğunda, veritabanından en alakalı parçayı bulup

```
LLM'e gönderecek sistem
qa_chain = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(model_name="gpt-4o"),
    chain_type="stuff",
    retriever=vector_db.as_retriever()
)
```

4. TEST SORUSU

```
soru = "Sistemdeki hata loglarının aciliyet sınıflandırması nasıl yapılır?"
cevap = qa_chain.invoke(soru)
print(cevap)
```

14.3. Kodun Çalışma Mantığı (Mühendislik Açıklaması)

1. **Text Splitter:** LLM'lerin bir "bağlam penceresi" (context window) limiti olduğu için, devasa dokümanları 1000 karakterlik küçük parçalara böleriz.
2. **Embeddings:** Metni, yapay zekanın anlayabileceği 1536 boyutlu bir sayı dizisine dönüştürürüz.
3. **Retriever (Geri Getirici):** Kullanıcının sorusu ile veritabanındaki metin parçaları arasındaki "kosinüs benzerliğini" (cosine similarity) hesaplayarak en alakalı parçayı buluruz.
4. **Chain:** Bulunan alakalı metin parçası + Kullanıcı sorusu, LLM'e gönderilir ve model bu dokümana dayanarak "halüsinasyon görmeden" cevap üretir.

BÖLÜM 15: AI MODELLERİNİ API İLE YÖNETMEK

Yapay zeka modellerini web arayüzlerinden (ChatGPT ekranı gibi) kullanmak, sadece manuel işler içindir. Gerçek mühendislik projelerinde modelleri kendi yazılımlarımıza, veritabanlarımıza veya otomasyon araçlarımıza bağlamak için **API (Application Programming Interface)** kullanırız.

15.1. API Anahtarı ve İstek Yapısı (Request Structure)

Bir modelle konuşmak için ona bir "istek" (request) göndeririz. Bu istek genellikle JSON formatındadır.

```
Python
import openai

client = openai.OpenAI(api_key="SENIN_API_ANAHTARIN")

response = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "system", "content": "Sen teknik bir döküman çevirmenisin."},
        {"role": "user", "content": "API nedir kısaca açıkla."}
    ],
    temperature=0.7 # Yaratıcılık düzeyi (0-1 arası)
)

print(response.choices[0].message.content)
```

15.2. JSON Çıktılarını İşlemek (Pars etme)

API'den gelen karmaşık verileri (JSON) kendi sistemimize entegre etmek için Python'un `json` kütüphanesi kullanılır. Özellikle veritabanına veri kaydedeceksek, modelden **"Strict JSON"** formatında çıktı istemek hayati önem taşır.

- **Prompt Stratejisi:** "Bana bu veriyi sadece JSON formatında ver, başka hiçbir açıklama yazma."
- **İşleme:** Gelen veriyi `json.loads()` ile bir Python sözlüğüne (dictionary) dönüştürüp değişkenlerde kullanabilirsin.

15.3. Rate Limit ve Hata Yönetimi

API kullanırken çok fazla istek atarsan sunucu seni geçici olarak engeller (Rate Limit). Mühendislik seviyesinde bir sistemde, mutlaka `try-except` blokları ve `time.sleep()` fonksiyonları ile bekleme süreleri tanımlanmalıdır.

BÖLÜM 16: SONUÇ VE GELECEK VİZYONU

Bu rehberle, yapay zekayı bir "araç" olmaktan çıkarıp, mühendislik süreçlerinizin merkezine yerleştirecek teknik altyapıyı tamamladık.

- **Özetle:** RAG mimarisi ile veri güvenliği sağladık, ileri prompt teknikleri ile modelin halüsinasyonlarını minimize ettik ve API entegrasyonu ile sistemlerinizi otomatize ettik.
- **Gelecek Vizyonu:** 2026 itibarıyla yapay zeka, "metin yazan bir bot" seviyesinden çıkıp "kendi başına karar alıp işlem yapan bir ajan" seviyesine evrilmiştir. Bu el kitabındaki teknikleri kullanarak sadece mevcut sorunları çözmekle kalmayacak, yeni nesil otonom sistemlerin mimarisi olacaksınız.

Unutmayın: Yapay zeka gelişir, araçlar değişir; ancak **"doğru soruyu sorma yeteneği"** ve **"çıktıyı doğrulama disiplini"** bir mühendisin sahip olabileceği en değerli yeteneklerdir.

Kitabın Tamamlandı!

Şu an elinde yaklaşık 30-32 sayfa civarında, tamamen teknik, standart sektör örnekleriyle dolu ve RAG sistemini test etmek için kusursuz bir veri seti içeren bir "Yapay Zeka El Kitabı" var.

1. Bunu Word dosyasında son kez düzenle.
2. PDF olarak dışa aktar.
3. Artık kendi kurduğun RAG sistemine bu PDF'i yükleyip, tüm bölümlere dair hazırladığımız o 160+ örnek soruyu sorarak sisteminin "IQ"sunu ölçebilirsin.

Başka eklememi istediğin bir detay var mı, yoksa bu halini "Final" olarak kabul edip bu serüveni noktalayalım mı?